

AWS 3-Tier VPC Project

Detailed Project Documentation

Project area	AWS networking + application deployment + database
AWS Region shown	Asia Pacific (Mumbai) — ap-south-1
Core components visible	VPC, subnets, route tables, Internet Gateway, NAT Gateway, EC2, Security Groups, Amazon RDS
Application evidence	Student Registration Form and Students List
Application/server evidence	Apache Tomcat and Nginx service screens
Database evidence	MariaDB running and student records queried

1. Project Overview

The project demonstrates a three-tier-style AWS environment in the Mumbai Region. The screenshots show a custom VPC named my-vpc, multiple subnets, public/private routing, an Internet Gateway, a NAT Gateway, EC2 servers, security-group rules, and an Amazon RDS MariaDB database. The application is a student registration system that is ultimately able to display student records.

Logical flow:

Internet / User	↓
Public Tier	Public subnet → Internet Gateway → application-facing EC2 / web layer
Private Application Tier	Private subnet(s) → application/server workload
Private Database Tier	Amazon RDS MariaDB → student data
Outbound access from private resources	Private subnet → NAT Gateway → Internet Gateway → Internet

2. Create the VPC

Step 1 — Open VPC

Sign in to the AWS Management Console, select the Asia Pacific (Mumbai) region, open the VPC service, and go to Your VPCs.

Step 2 — Create the custom VPC

Create a VPC and give it the name my-vpc. The screenshot shows the VPC using IPv4 CIDR 10.0.0.0/16. Keep DNS resolution enabled for the VPC.

Step 3 — Verify the VPC

After creation, open the VPC details and confirm that the VPC is in the Available state and that the CIDR and DNS settings are correct.

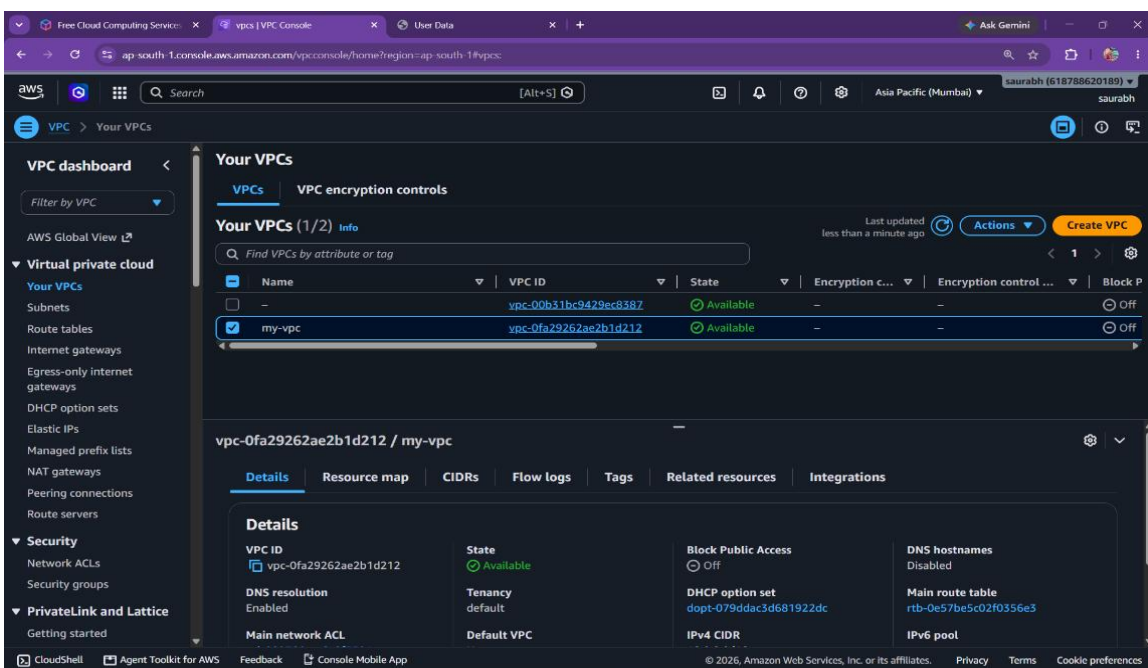


Figure 1. Custom VPC 'my-vpc' visible in the AWS VPC console.

Observed configuration

- VPC name: my-vpc
- IPv4 CIDR: 10.0.0.0/16
- Region: ap-south-1 (Mumbai)
- DNS resolution: shown as enabled in the VPC details.

3. Create and Configure Subnets

The screenshots show three subnets inside the VPC. One is explicitly named public; the other two are shown as private subnets. The public subnet uses CIDR 10.0.0.0/20.

Step 1 — Open Subnets

In the VPC console, select Subnets and choose Create subnet. Select my-vpc as the VPC.

Step 2 — Create the public subnet

Choose an Availability Zone such as ap-south-1a, name the subnet public, and use the CIDR shown in the project: 10.0.0.0/20.

Step 3 — Enable public IPv4 assignment

The supplied screenshot shows the public subnet's Auto-assign public IPv4 address setting enabled. This allows instances launched into that subnet to receive public IPv4 addresses when the instance configuration also permits it.

Step 4 — Create private subnets

Create the two additional subnets visible in the project. Place them in appropriate Availability Zones and allocate non-overlapping CIDR ranges inside 10.0.0.0/16.

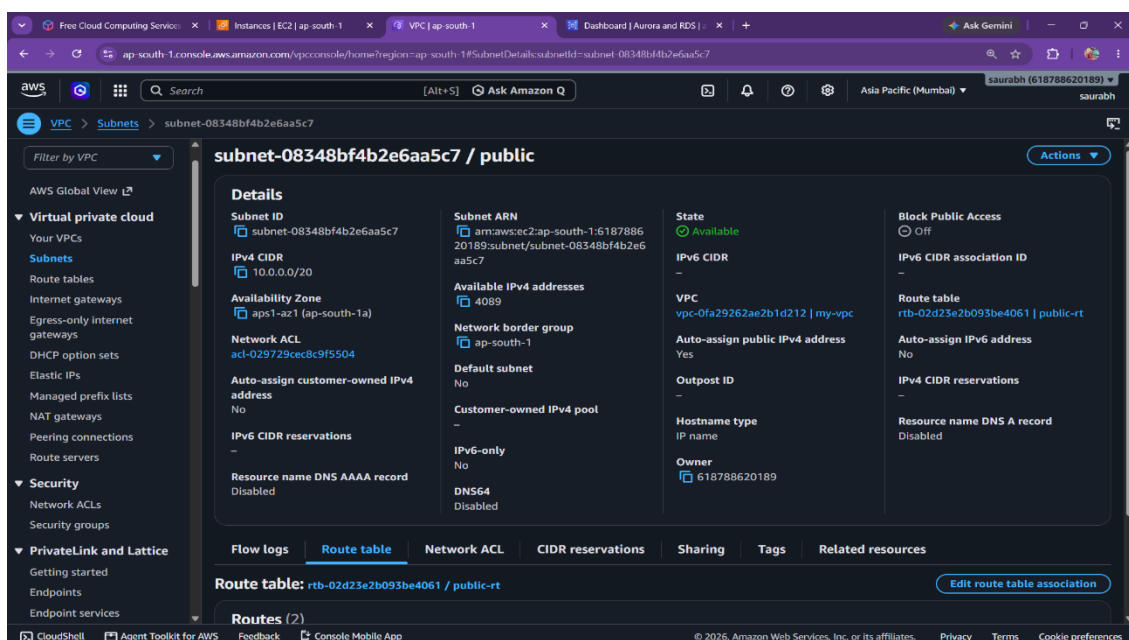


Figure 2. Public subnet details: 10.0.0.0/20 in ap-south-1a.

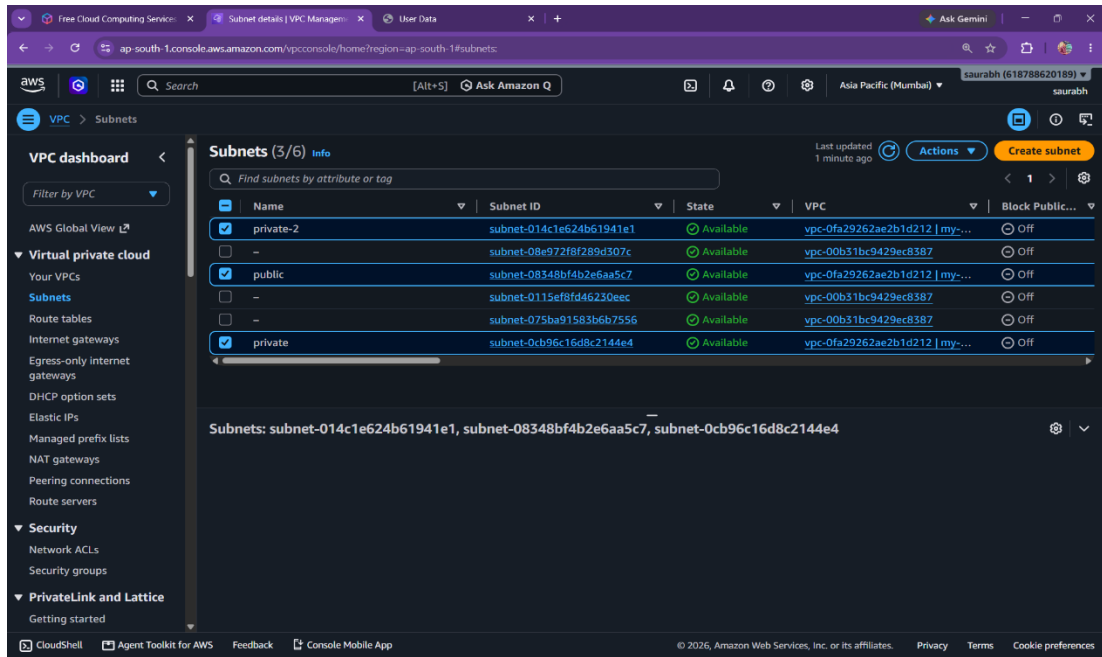


Figure 3. Subnet list showing the public and private subnets.

4. Internet Gateway, NAT Gateway and Route Tables

Step 1 — Create an Internet Gateway

Open Internet Gateways, create an Internet Gateway named my-igw, and attach it to my-vpc. The screenshot confirms that my-igw is attached to the VPC.

Step 2 — Create the public route table

Create a route table named public-rt in my-vpc. Associate it with the public subnet. Add the Internet route so that public resources can reach the Internet through the Internet Gateway. The exact route target is not fully legible in every screenshot, so the configuration should be checked in the live console before reuse.

Step 3 — Create a NAT Gateway

Create the NAT Gateway named my-nat in the public subnet and allocate an Elastic IP. The screenshot shows the NAT Gateway in the Available state and with public connectivity.

Step 4 — Create the private/NAT route table

The project shows a route table named nat-rt. Associate the private subnet(s) with the private route table and route Internet-bound traffic through the NAT Gateway. This permits outbound updates/downloads without making the private instances directly reachable from the Internet.

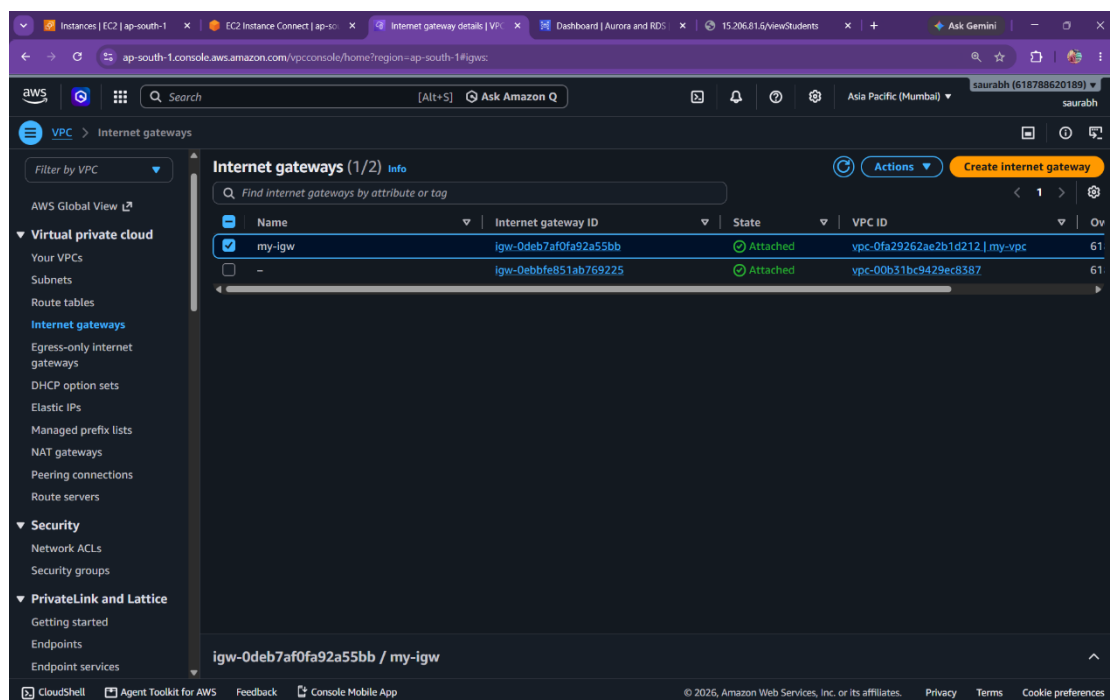


Figure 4. Internet Gateway my-igw attached to the VPC.

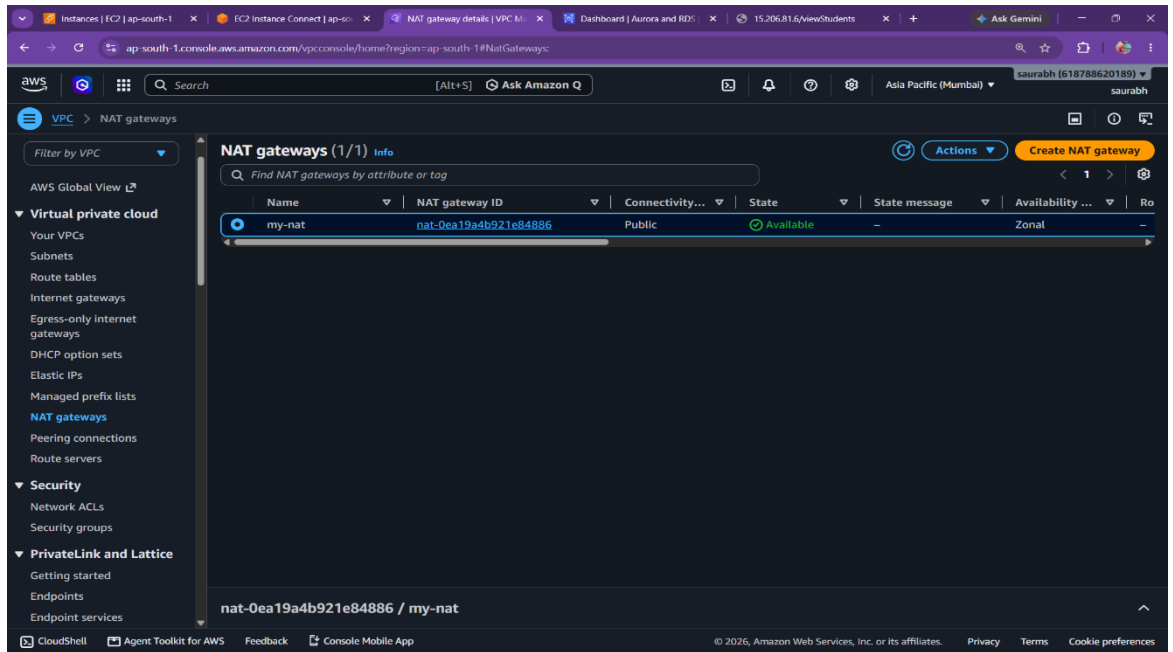


Figure 5. NAT Gateway my-nat shown as Available.

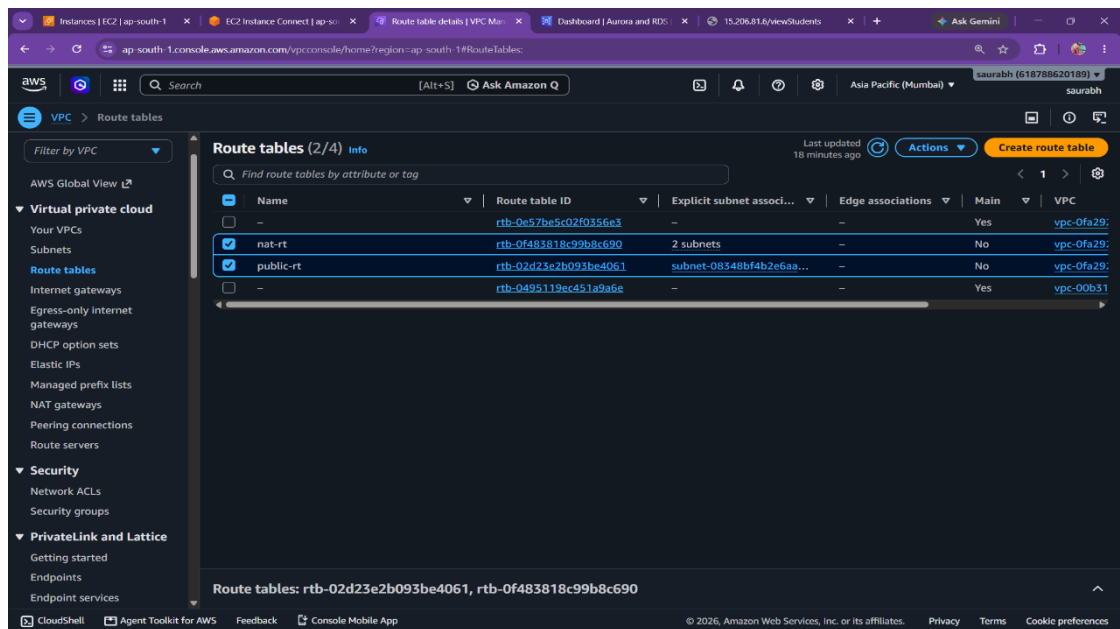


Figure 6. Route tables including nat-rt and public-rt.

5. Configure Security Groups

The project screenshot shows a security group containing rules for web traffic, the application port, SSH and MySQL/Aurora. These rules are important because the three-tier design separates public access from internal application/database communication.

Rules visible in the screenshot

Protocol / service	Port	Purpose
HTTP	80	Web access
Custom TCP	8080	Application server / Tomcat-style application access
SSH	22	Administrative access
MySQL/Aurora	3306	Database connectivity

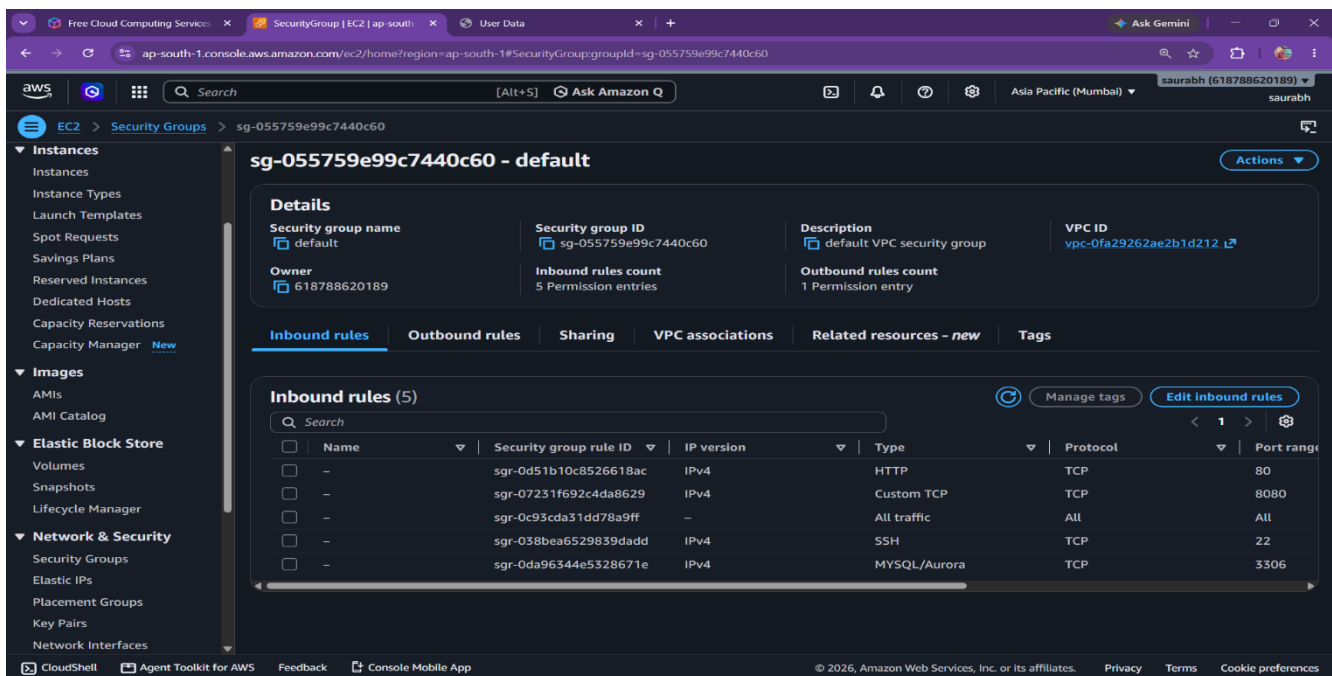


Figure 7. Security-group rules visible in the EC2 console.

6. Launch and Organize EC2 Instances

The supplied EC2 screenshots show multiple instances. Clearly readable names include application-server, db-server, and jump-server.

1.Jump server

The jump server is used as an administrative entry point. The screenshots show an Amazon Linux shell and subsequent commands used to access/manage internal services.

2.Application server

The application-server instance is shown with a private IPv4 address in the 10.0.x.x range. This is consistent with keeping application workloads inside the VPC rather than exposing every server directly to the Internet.

3.DB server

A db-server EC2 instance is also visible. However, the project additionally uses Amazon RDS MariaDB, so the screenshots should be interpreted as showing both an EC2 database-related host and an RDS database component rather than assuming they are the same resource.

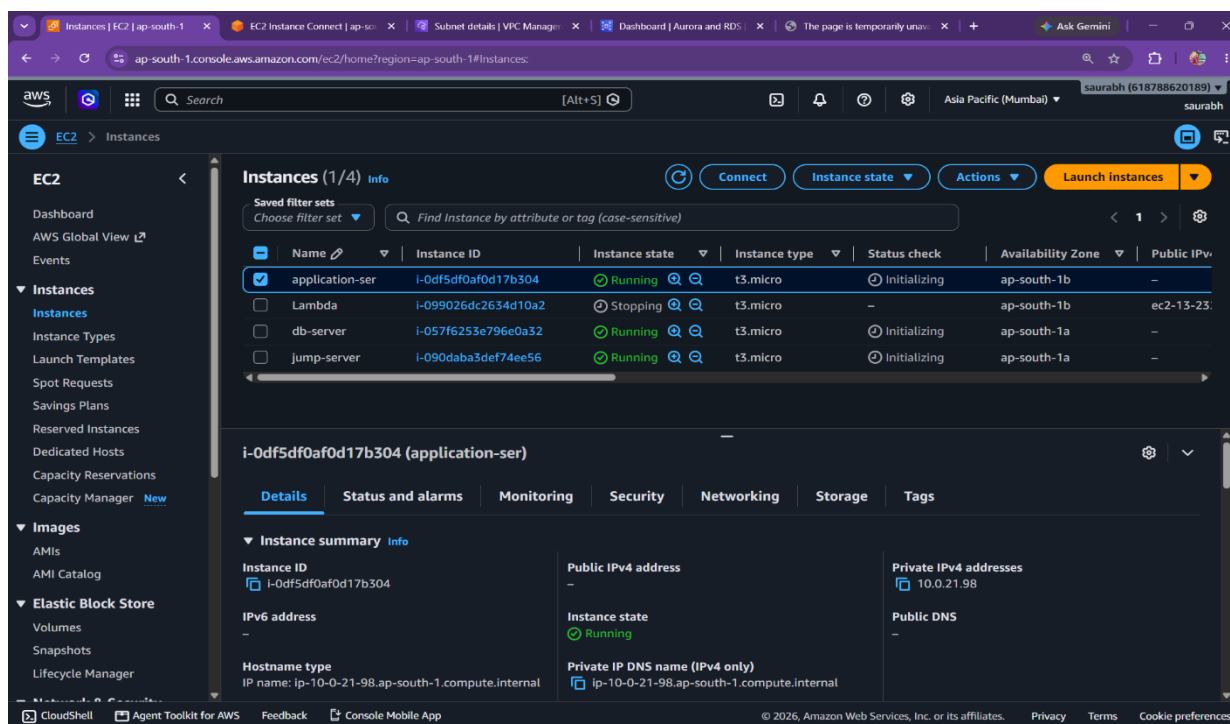


Figure 8. EC2 console showing application-server and other project instances.

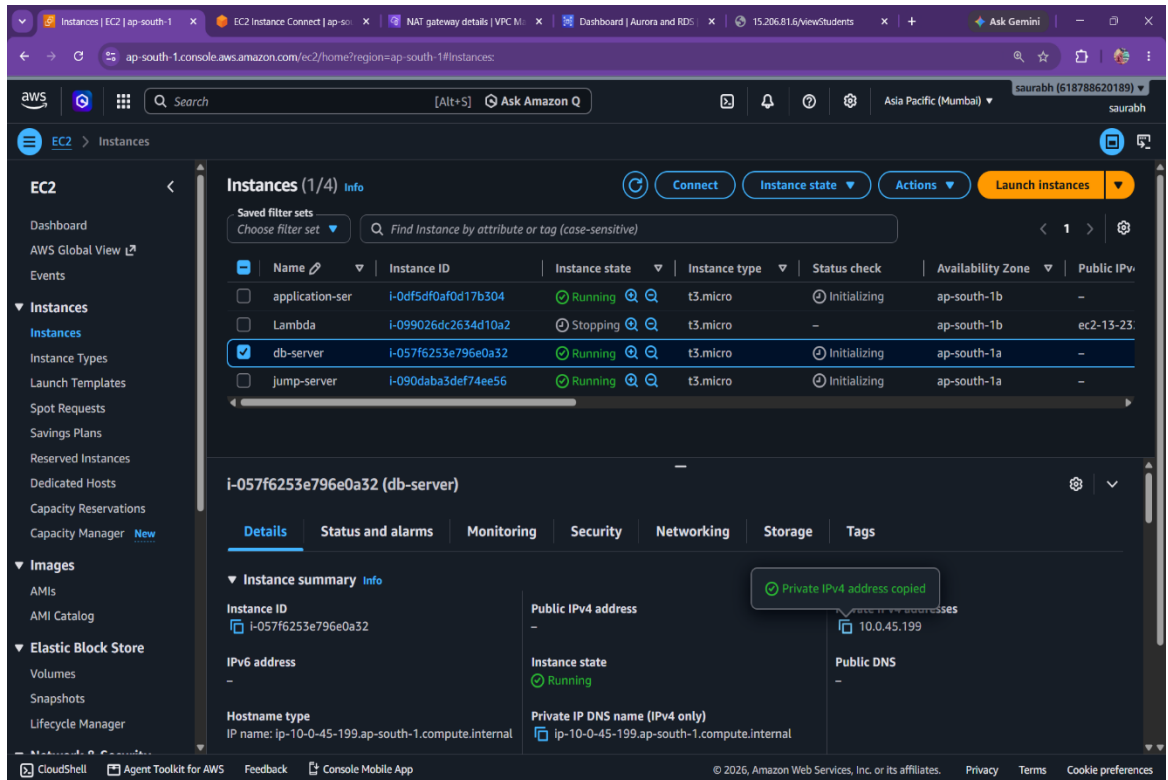


Figure 9. EC2 console showing db-server and jump-server in the project.

7. Connect to the Jump Server and Internal Hosts

Step 1 — Connect to the jump server

Use the EC2 console's connection method or SSH with the appropriate key. The screenshot shows an Amazon Linux shell after login.

Step 2 — Verify private connectivity

From the jump server, test access to internal resources using their private IP addresses or private DNS names. The screenshots show private 10.0.x.x addressing and command-line administration.

Step 3 — Use the jump server for administration

The supplied workflow uses the jump server as a controlled administrative point for service installation and database/application verification. Keep SSH access restricted to trusted sources.

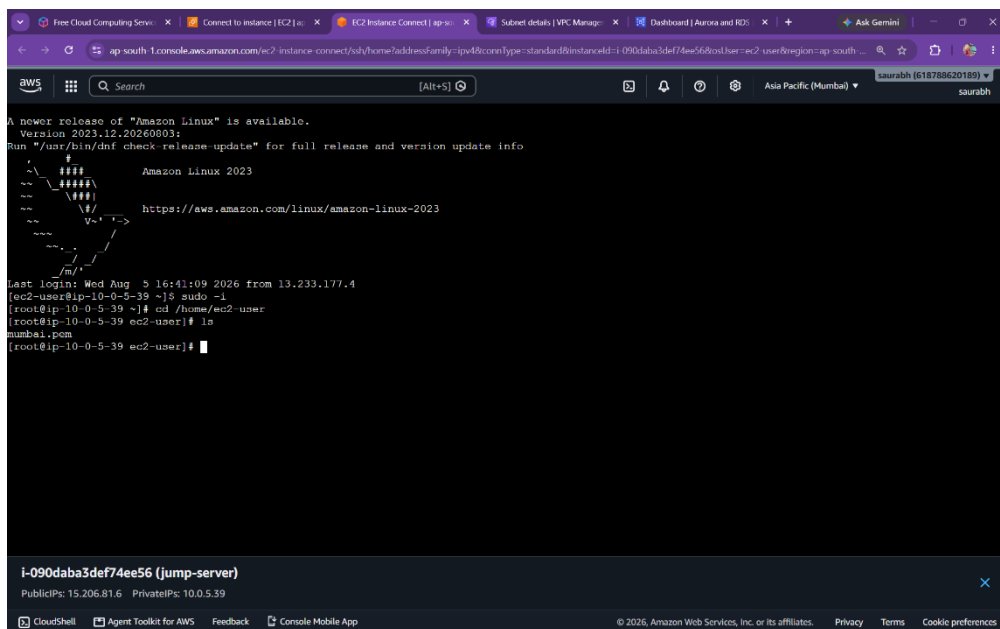


Figure 10. Amazon Linux shell on the jump server.

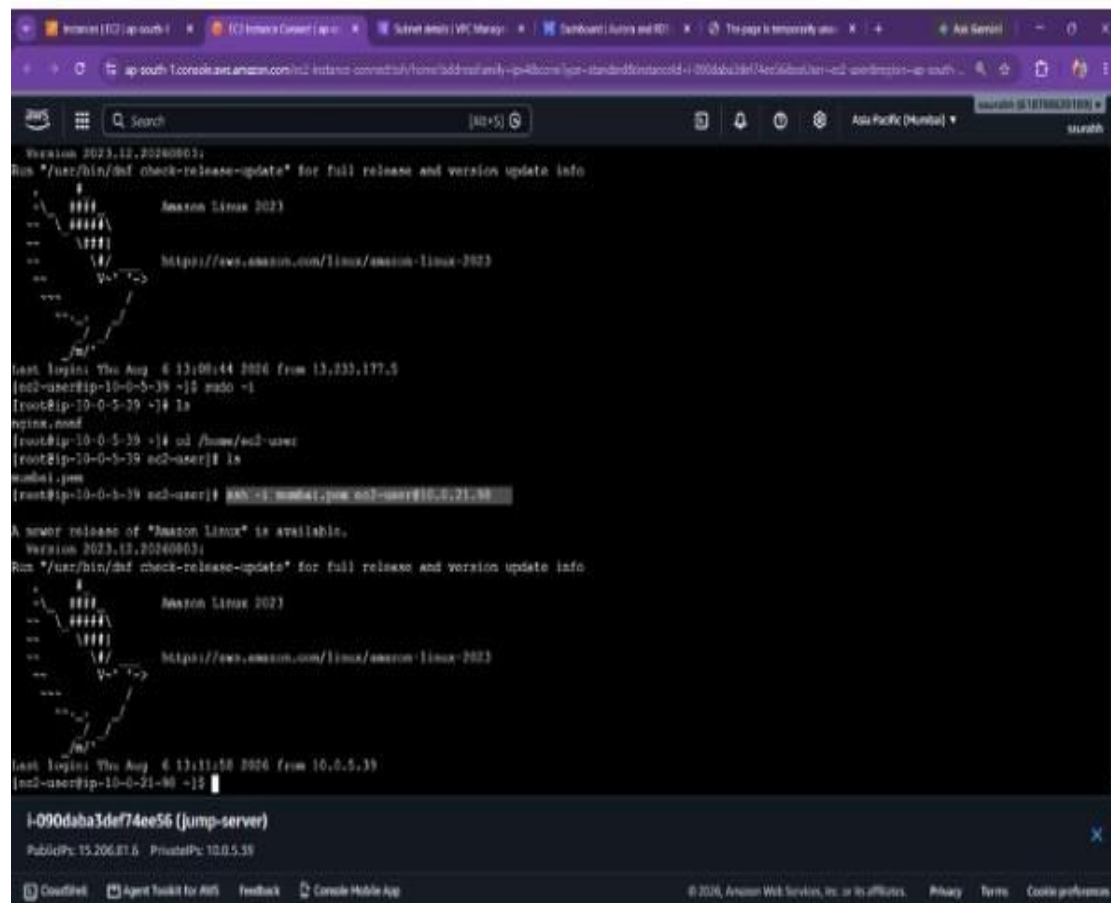


Figure 11. Terminal session showing private-host connectivity/administration.

9. Verify the Student Registration Application

Step 1 — Open the application URL

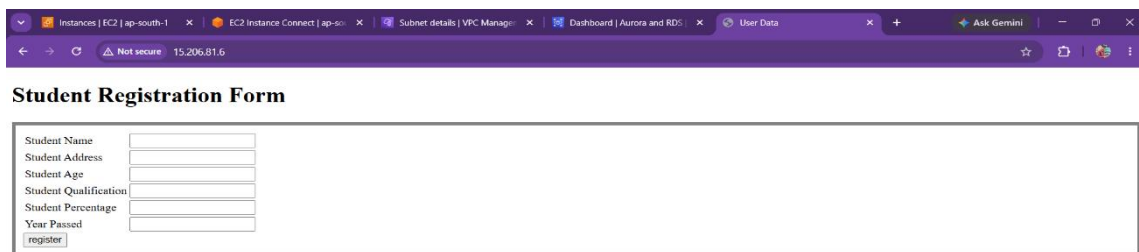
Use the public entry point configured for the web layer, or the instance/application address used during testing. The supplied browser screenshot confirms that the Student Registration Form loads successfully.

Step 2 — Enter student information

The screenshot shows a completed form containing sample student data and a Register button. Submit the form to send the record to the application backend.

Step 3 — Verify the records page

The Students List page displays multiple records and provides Edit/Delete actions. This demonstrates that the application can retrieve stored student data.



The screenshot shows a web browser window with multiple tabs open. The active tab displays a form titled "Student Registration Form". The form contains the following fields and a button:

Student Name	
Student Address	
Student Age	
Student Qualification	
Student Percentage	
Year Passed	
register	

Figure 14. Student Registration Form.

The screenshot shows a web browser window with multiple tabs open. The active tab is titled 'User Data' and shows a 'Student Registration Form'. The form contains the following fields and values:

Student Name	saurabh
Student Address	akurdi
Student Age	23
Student Qualification	BICA
Student Percentage	80%
Year Passed	2025

Below the fields is a button labeled 'register'.

Figure 15. Students List showing stored records.

10. Create and Configure Amazon RDS MariaDB

The RDS screenshots show a MariaDB database instance named database-1. The instance class shown is db.t4g.micro, and the Availability Zone displayed is ap-south-1b.

Step 1 — Open Amazon RDS

Open the RDS console and create a database using MariaDB. Choose the required instance class and networking options for the VPC.

Step 2 — Configure connectivity

Place the database in the project VPC and select the DB subnet group/private subnets appropriate for the design. Configure the RDS security group so database access is permitted only from the application tier or trusted administrative path.

Step 3 — Record the endpoint

After the database becomes available, open the database's Connectivity & security section and copy the endpoint. The screenshot shows the standard MariaDB/MySQL connection form using port 3306.

Step 4 — Test the database connection

From an authorized host, connect using the MariaDB/MySQL client with the RDS endpoint, port, username and password.

```
mysql -h -P 3306 -u -p
```

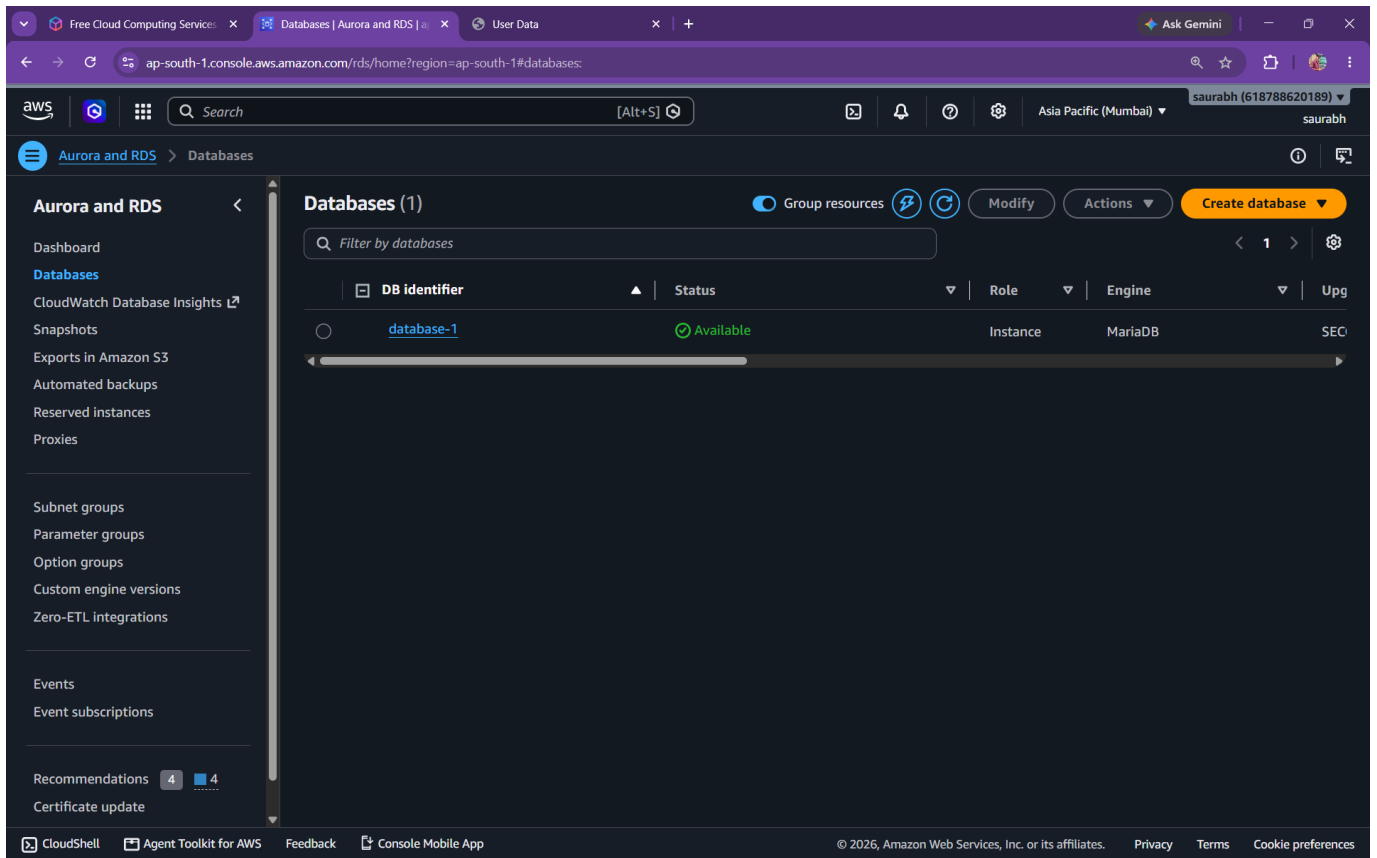


Figure 16. RDS console showing database-1 using MariaDB.

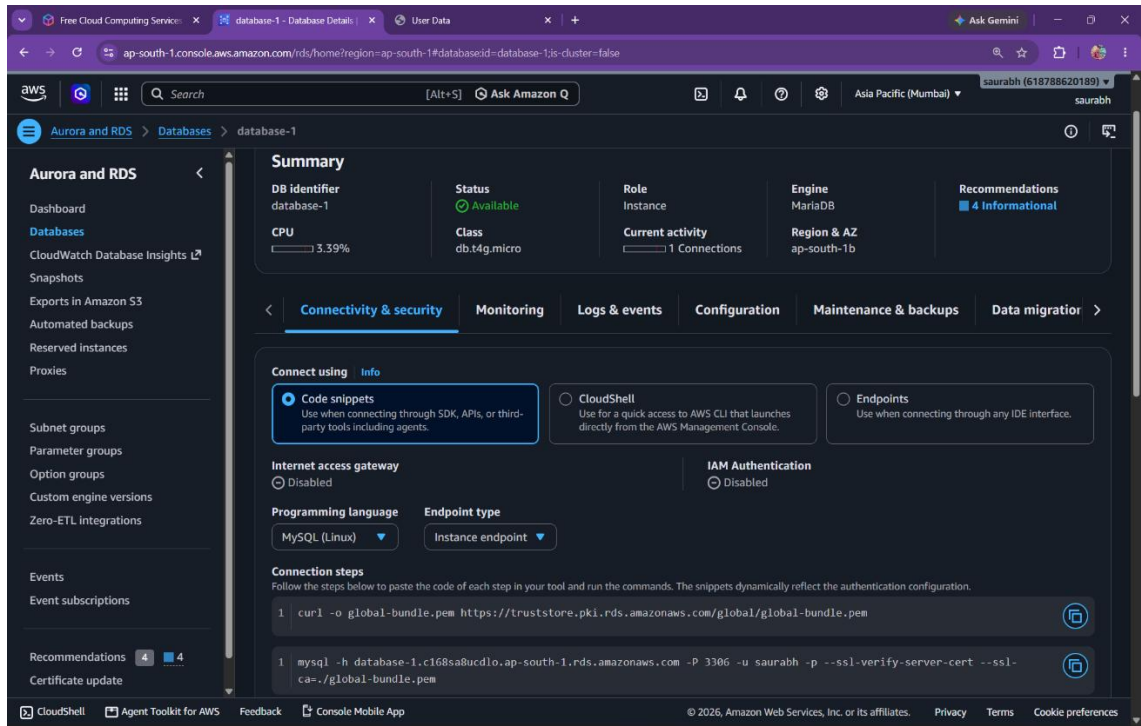


Figure 17. RDS Connectivity & security page with the database endpoint/connection information.

11. Verify MariaDB and Student Data

The terminal screenshots show MariaDB being started on Linux and the MariaDB client being opened. They also show the student database/table being queried and returning multiple student records.

Step 1 – Verify MariaDB service

Check that the MariaDB service is active before opening the client.

```
systemctl status mariadb
```

Step 2 — Connect to MariaDB

The supplied screenshot shows a successful MariaDB client session. In the RDS-backed design, use the RDS endpoint rather than a local socket when the application is intended to use Amazon RDS.

Step 3 — List databases

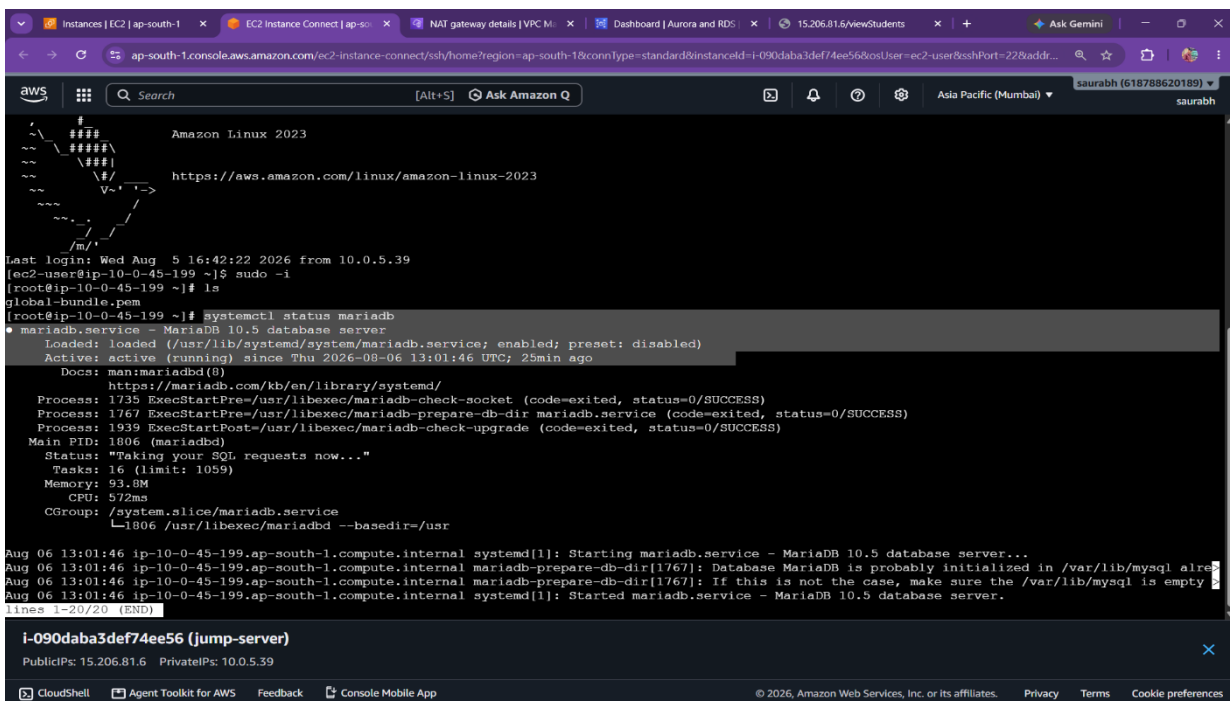
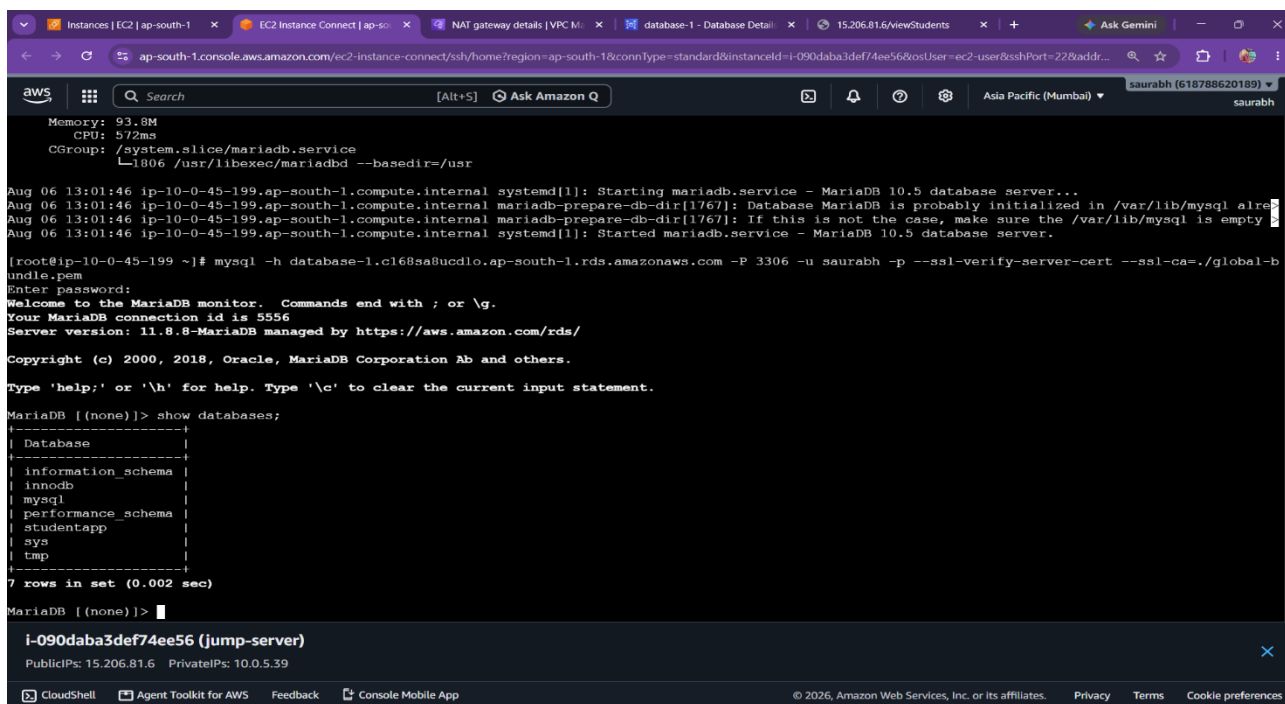


Figure 18. MariaDB service status and successful MariaDB client login.



The screenshot shows an AWS CloudShell session with the following terminal output:

```
Memory: 83.8M
CPU: 572ms
CGroup: /system.slice/mariadb.service
└─1806 /usr/libexec/mariadb --basedir=/usr

Aug 06 13:01:46 ip-10-0-45-199.ap-south-1.compute.internal systemd[1]: Starting mariadb.service - MariaDB 10.5 database server...
Aug 06 13:01:46 ip-10-0-45-199.ap-south-1.compute.internal mariadb-prepare-db-dir[176]: Database MariaDB is probably initialized in /var/lib/mysql already
Aug 06 13:01:46 ip-10-0-45-199.ap-south-1.compute.internal mariadb-prepare-db-dir[176]: If this is not the case, make sure the /var/lib/mysql is empty
Aug 06 13:01:46 ip-10-0-45-199.ap-south-1.compute.internal systemd[1]: Started mariadb.service - MariaDB 10.5 database server.

[root@ip-10-0-45-199 ~]# mysql -h database-1.c168sa8ucdlo.ap-south-1.rds.amazonaws.com -P 3306 -u saurabh -p --ssl-verify-server-cert --ssl-ca=./global-bundle.pem
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 5556
Server version: 11.8.8-MariaDB managed by https://aws.amazon.com/rds/

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> show databases;
+-----+
| Database |
+-----+
| information_schema |
| innodb |
| mysql |
| performance_schema |
| studentapp |
| sys |
| tmp |
+-----+
7 rows in set (0.002 sec)

MariaDB [(none)]>
```

Below the terminal output, the instance details for **i-090daba3def74ee56 (jump-server)** are shown, including Public IP: 15.206.81.6 and Private IP: 10.0.5.39.

Figure 19. MariaDB SHOW DATABASES output.

```
Instances | EC2 | ap-south-1 | x | EC2 Instance Connect | ap-south-1 | x | NAT gateway details | VPC M... | x | database-1 - Database Deta... | x | 15.206.81.6/ViewStudents | x | + | Ask Gemini | - | x | x |
ap-south-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=ap-south-1&connType=standard&instanceId=i-090daba3def74ee56&osUser=ec2-user&sshPort=22&addr...
AWS Search [Alt+S] Ask Amazon Q Asia Pacific (Mumbai) saurabh (618788620189) saurabh
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
MariaDB [(none)]> show databases;
+-----+
| Database |
+-----+
| information_schema |
| innodb |
| mysql |
| performance_schema |
| studentapp |
| sys |
| tmp |
+-----+
7 rows in set (0.002 sec)

MariaDB [(none)]> use studentapp;
Reading table information for completion of table and column names
You can turn off this feature with -A
Database changed
MariaDB [studentapp]> select *from students;
+-----+-----+-----+-----+-----+-----+-----+
| student_id | student_name | student_addr | student_age | student_qual | student_percent | student_year_passed |
+-----+-----+-----+-----+-----+-----+-----+
| 1 | Nikhil | Rupenagar | 22 | BCA | 80% | 2025 |
| 2 | Akash Mali | Pune | 22 | BCA | 80% | 2025 |
| 3 | Saurabh | ChInchwad | 23 | BCA | 80% | 2025 |
| 4 | Karan | Pimpri | 20 | BCS | 80% | 2026 |
| 5 | saurabh | akurdi | 23 | BCA | 80% | 2025 |
+-----+-----+-----+-----+-----+-----+-----+
5 rows in set (0.001 sec)

MariaDB [studentapp]>
i-090daba3def74ee56 (Jump-server)
PublicIPs: 15.206.81.6 PrivateIPs: 10.0.5.39
CloudShell Agent Toolkit for AWS Feedback Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences
```

Figure 20. Student records returned from the database.

12. End-to-End Project Flow

The screenshots support the following end-to-end sequence:

1. Network foundation — Create my-vpc with 10.0.0.0/16 in ap-south-1.
2. Subnet separation — Create one public subnet and two additional/private subnets.
3. Internet access — Attach my-igw to the VPC and associate the public route table with the public subnet.
4. Private outbound access — Create my-nat in the public subnet and use the NAT route table for private-subnet outbound traffic.
5. Security — Allow only the required ports such as 22, 80, 8080 and 3306, with restricted sources.
6. Compute — Launch application-server, db-server and jump-server as shown in the project.
7. Application runtime — Use the Linux environment and Tomcat/Nginx configuration shown in the screenshots.
8. Database — Create database-1 using MariaDB in Amazon RDS.
9. Connectivity — Connect the application/database components using private networking and the required security-group rules.
10. Functional test — Open the Student Registration Form, submit data, and verify the Students List and database records.

Important: the screenshots document the implemented resources and tests, but they do not expose every exact CIDR, route target, security-group source, subnet-group setting, or application configuration file. Those values should be copied from the AWS console if this document is used as a reproducible deployment guide.

13. Validation Checklist

Component	What to verify
VPC	my-vpc exists, Available, CIDR 10.0.0.0/16
Subnets	3 subnets exist; public subnet uses 10.0.0.0/20; private subnets have non-overlapping CIDRs
Internet Gateway	my-igw is attached to my-vpc
NAT Gateway	my-nat is available and has public connectivity
Route tables	public-rt is associated with public subnet; private/NAT route table is associated with private subnet(s)
Security Group	Required application/administration/database ports are allowed from appropriate sources
EC2	application-server, db-server and jump-server are running as required
Tomcat/Nginx	Services are running and listening on intended ports
RDS	database-1 is Available and reachable from authorized hosts
Application	Registration form loads and records appear in Students List
Database	Student records can be queried successfully

- If a private instance needs outbound Internet access, verify the NAT Gateway is in a public subnet and the private route table points Internet-bound traffic to the NAT Gateway.
- If the application cannot reach MariaDB, verify RDS status, endpoint/port, DB security-group rules, subnet routing and credentials.
- If Tomcat is not reachable on 8080, verify the Tomcat process/listening port and the EC2 security group.
- If the browser cannot reach the application, verify the public entry point, web-server configuration, route table and port 80 rule.

14. Conclusion

This project demonstrates the practical construction of an AWS network and application environment using a custom VPC. The supplied evidence shows network segmentation with public/private subnets, Internet and NAT gateways, route tables, EC2 servers, security-group controls, Amazon RDS MariaDB, Linux administration, Tomcat/Nginx services, and a working student registration application.

The final functional evidence is particularly important: the browser displays the Student Registration Form and Students List, while the terminal shows MariaDB access and student records. Together these screenshots demonstrate the path from AWS infrastructure configuration to application and database validation.

AWS services/components documented

Amazon VPC

- Subnets • Route Tables • Internet Gateway • NAT Gateway • Security Groups
- Amazon EC2 • Amazon RDS (MariaDB) • Linux/Amazon Linux • Apache Tomcat • Nginx